Observability Dashboard Guide

This document provides a comprehensive overview of the **PBI Beacon Observability Dashboard**. It is divided into two parts:

1. **Business Overview:** For stakeholders and business users to understand the metrics.
2. **Technical Reference:** For developers to understand the underlying code and implementation.

Part 1: Business Metrics & Dashboard Guide

Executive Summary

The Observability Hub provides real-time transparency into the AI Assistant's performance. It allows stakeholders to monitor adoption (volume), resource consumption (tokens), operational health (latency), and financial impact (cost).

The 8 Key Performance Indicators (Cards)

The dashboard displays 8 primary metric cards at the top. Here is a detailed explanation of each.

1. Total Requests

- **Definition:** The total number of interactions (questions/turns) users have had with the AI.
- **Why it matters:** This is the primary metric for **Adoption**. High numbers indicate the tool is being used frequently.
- **Sub-metric:** The dashboard also displays the breakdown of "Successful" vs. "Failed" requests, giving a quick view of reliability.

2. Input Tokens

- **Definition:** The amount of text sent *to* the AI model. This includes the user's question plus any background documents or search results provided to the AI to help it answer.
- **Why it matters:** Represents the "Reading Load" of the system. High input tokens mean the AI is processing large amounts of context (e.g., reading long PDFs) to answer questions.

3. Output Tokens

- **Definition:** The amount of text *generated* by the AI model in its response.

- **Why it matters:** Represents the "Writing Load." Long, detailed answers increase this metric.
- **Note:** Output tokens are typically more expensive per unit than input tokens.

4. Total Tokens

- **Definition:** The sum of Input Tokens + Output Tokens.
- **Why it matters:** This is the standard unit of measurement for **AI Workload**. It combines both reading and writing volume to show total processing throughput.

5. Avg Latency

- **Definition:** The average time it takes for the AI to generate a complete answer (measured in seconds or milliseconds).
- **Why it matters:** Key metric for **User Experience**. Lower is better. Spikes in latency can indicate network issues or that the model is struggling with complex tasks.

6. Input Cost

- **Definition:** The estimated dollar cost associated with processing all Input Tokens.
- **Why it matters:** Helps budget for the "context" part of the AI operations (uploading and reading documents).

7. Output Cost

- **Definition:** The estimated dollar cost associated with generating all answers.
- **Why it matters:** Helps budget for the "generation" part of the AI operations.

8. Total Cost

- **Definition:** The cumulative financial cost of running the AI service (`Input Cost + Output Cost`).
- **Why it matters:** Critical for **ROI analysis** and budget tracking. It provides a real-time view of spend without waiting for monthly billing cycles.

Detailed Analytics Views

Below the summary cards, the dashboard offers four specialized views for deeper analysis.

1. Overview (System Health)

- **Purpose:** A high-level health check of the system.
- **Key Metrics:**
  - **Success Rate:** Percentage of error-free interactions (Target: >95%).
  - **Average Tokens/Request:** Complexity score. Higher numbers mean "heavier" questions.
  - **Cost per Request:** The average price of a single interaction (e.g., $0.03). This is vital for calculating the cost-benefit of the tool.

2. By Model

- **Purpose:** Break down usage by the specific LLM used (e.g., GPT-4 vs. GPT-3.5).
- **Why it matters:** Different models have different price points. This view helps ensure we aren't using the most expensive model for simple tasks, allowing for cost optimization.
- **Columns Explained:**
  - **Model:** The specific AI model name (e.g., `gpt-4`).
  - **Requests:** Count of times this specific model was used.
  - **Input Tokens:** Total context read by this model.
  - **Output Tokens:** Total text generated by this model.
  - **Total Tokens:** Sum of input + output workload for this model.
  - **Cost:** Total money spent on this specific model.
  - **Avg Latency:** Average speed for this model.

3. Hourly Breakdown

- **Purpose:** A timeline showing activity volume over the last 24 hours.

- **Why it matters:** Identifies **Peak Usage Hours**. This helps in planning maintenance windows or understanding user work patterns (e.g., spikes at 9 AM).
- **Columns Explained:**
  - **Hour:** The specific time block (e.g., "2024-03-20 09:00").
  - **Requests:** Number of questions asked during that hour.
  - **Cost:** Total money spent during that hour.

4. Recent Requests

- **Purpose:** A detailed audit log of the last 50 individual interactions.
- **Why it matters:** Essential for **Auditing and Debugging**. If a user reports an issue, this log allows admins to pinpoint exactly which request failed and why.
- **Columns Explained:**
  - **Timestamp:** Exact date and time the question was asked.
  - **Model:** Which AI model handled the request.
  - **Tokens:** The breakdown of workload (Total (Input / Output)).
  - **Cost:** The precise cost calculated for this single interaction.
  - **Latency:** Time taken to generate the answer.
  - **Status:** "Success" (Green) or "Failed" (Red).

Part 2: Technical Architecture & Implementation

(For Developers & Technical Stakeholders)

This section details the code responsible for collecting the metrics described above.

1. Request Tracking Engine

File: `backend/Agents/observability.py`

The system uses a thread-safe singleton class `ObservabilityTracker` to record metrics in memory. This ensures high performance without blocking the main application flow.

```python
@dataclass
class LLMRequest:
    """Represents a single LLM request with all relevant metrics."""
    timestamp: str
    model: str
    input_tokens: int
    output_tokens: int
    total_tokens: int
    cost: float
    # ... other metrics

class ObservabilityTracker:
    def __init__(self):
        self._requests: List[LLMRequest] = []
        self._lock = threading.Lock()

    def track_request(self, request_data):
        with self._lock:
            self._requests.append(request_data)
```

2. Automated Token Counting

File: `backend/Agents/observability.py`

We rely on the `litellm` library's callback system to capture precise token usage reported by the AI provider (e.g., Azure OpenAI) immediately after a request completes.

```python
def litellm_success_callback(kwargs, completion_response, start_time, end_time):
    # 1. Calculate Latency
    latency_ms = (end_time - start_time) * 1000

    # 2. Extract Real Token Usage from API Response
    usage = getattr(completion_response, "usage", None)
    if usage:
        input_tokens = getattr(usage, "prompt_tokens", 0)
        output_tokens = getattr(usage, "completion_tokens", 0)

    # 3. Log to Tracker
    observability_tracker.track_request(...)
```

3. Cost Calculation Strategy

File: `backend/Agents/observability.py`

Cost is calculated dynamically using a 4-layer fallback strategy to ensure reliability:

1. **LiteLLM Database:** Query the library's internal pricing database (primary source).
2. **Cache:** Check if we've already fetched pricing for this model.
3. **Web Fetch:** Attempt to retrieve pricing from external sources if unknown.
4. **Fallback:** Use a conservative estimate (GPT-4 pricing) to ensure costs are not under-reported.

```python
def calculate_cost(self, model, input_tokens, output_tokens):
    # Layer 1: LiteLLM
    try:
        in_cost, out_cost = litellm.cost_per_token(model=model, ...)
        return in_cost + out_cost
    except:
        # Layer 2: Cache / Web Fetch / Fallback
        # ... fallback logic ...
```

4. Frontend Data Synchronization

File: `myf/src/lib/api.ts`

The React frontend polls the backend every 5 seconds. It implements a failover mechanism: if the Python backend is offline (e.g., during demos), it switches to `localStorage` to keep the dashboard functional.

```javascript
export async function getObservabilityStats() {
  try {
    // 1. Try Backend
    const response = await fetch(`${API_BASE_URL}/observability/stats`);
    return response.json();
  } catch (error) {
    // 2. Fallback to Local Browser Storage
    console.warn("Using local data mirror");
    return loadObservabilityStats();
  }
}
```